

Improving XML Visualization Without Cross-Origin Issues

[1. Introduction](#)

[2. The Cross-Origin Restriction Problem](#)

[2.1 What Is Cross-Origin Access?](#)

[2.2 Symptom](#)

[3. Existing Workaround \(Local HTTP Server\)](#)

[4. Solution Options](#)

[4.1 Embedding the XSL Directly into the XML](#)

[4.2 Generating a Self-Contained HTML Report](#)

[5. References](#)

nt : Context and additional notes on pages 78, 97 and 98 of Notebook 2 - Sept 2024

1. Introduction

In many data processing and analysis pipelines, XML (eXtensible Markup Language) is used for structured data representation, often accompanied by an XSL (eXtensible Stylesheet Language) file to define its visual layout. While this approach works well in controlled environments, it presents usability issues in modern web browsers due to **cross-origin restrictions**.

This report explores methods to resolve this limitation while maximizing usability for non-technical end users.

2. The Cross-Origin Restriction Problem

2.1 What Is Cross-Origin Access?

Modern browsers implement strict **Cross-Origin Resource Sharing (CORS)** policies to protect users from potential security vulnerabilities. Under these policies, a file loaded using the `file://` protocol is considered a separate security origin. As a result, XML files referencing external XSL files via relative paths cannot load stylesheets when opened directly from the local file system

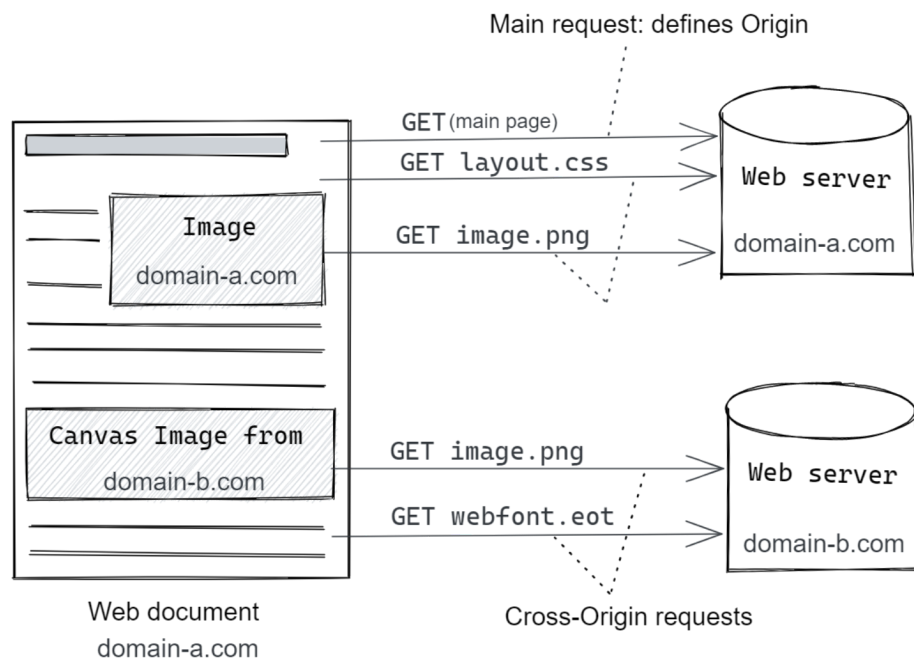


Figure 1: Cross Origin Access example

2.2 Symptom

When opening an XML file like `Pablo_20250410_0_GA.xml` directly in a browser, users encounter a blank page. The browser's Developer Tools report the following:

```
Unsafe attempt to load URL
file:///.../Style/general_analysis.xml from frame with URL
file:///.../Pablo_20250410_0_GA.xml. 'file:' URLs are treated as
unique security origins.
```

3. Existing Workaround (Local HTTP Server)

A common workaround is to serve the XML and its dependencies using a local HTTP server:

This resolves the issue since all files are served under a single HTTP origin, bypassing browser restrictions. **Not suitable for end users:** Requiring users to install Python and run terminal commands is not practical for non-technical audiences.

```
cd C:\Users\pablo\Documents\InputLog\Pablo\2025-04-10_0\output

python -m http.server 8000

http://localhost:8000/Pablo_20250410_0_GA.xml
```

inputlog

General Analysis

Meta Information

Logfile

Pablo_0.idfx

Main Document

WordLog_Pablo_20250410075946.docx

Log Creation

10/04/25 07:59:56.417

Log GUID

aac2d3dc-5caf-4542-ab6f-76824a010ecc

Loggings Program Version Number

9.5.0.5

Analysis Creation

10/04/25 08:05:51

Analysis GUID

760754c8-e86e-4108-acb8-1f1453efe4ba

Analysis Program Version Number

9.5.0.5

Session Identification

Participant

Pablo

Text Language

EN

Age

25

#Id	Event Type	Output	Position	DocLength	Character Production	StartTime	StartClock	EndTime	EndClock	ActionTime	PauseTime	PauseLocation	IntervalFixedSize	IntervalFixedNumber	X	Y
0	focus	new 1 - Notepad++			0	0	00:00:00	0	00:00:00	0	0	CHANGE	1	0		
1	mouse	Movement			0	0	00:00:00	656	00:00:00	656	0	INITIAL	1	0	751	376
2	focus	WordLog MainDoc			0	656	00:00:00	656	00:00:00	0	0	CHANGE	1	1		
3	mouse	LEFT Click			0	719	00:00:00	844	00:00:00	125	63	INITIAL	1	1	751	376
4	mouse	Movement			0	906	00:00:00	1359	00:00:01	453	62	MOUSE	1	1	662	10
5	mouse	LEFT Click			0	1562	00:00:01	2156	00:00:02	594	203	MOUSE	1	1	662	10
6	mouse	Movement			0	1594	00:00:01	2140	00:00:02	546	562	MOUSE	1	1	2653	0
7	mouse	Movement			0	2172	00:00:02	2922	00:00:02	750	32	MOUSE	1	1	2596	321
8	mouse	LEFT Click			0	3015	00:00:03	3140	00:00:03	125	93	MOUSE	1	1	2596	321
9	mouse	Movement			0	3703	00:00:03	3812	00:00:03	109	563	MOUSE	1	1	2052	342
10	keyboard	LALT	0	1	1	4031	00:00:04	7772	00:00:07	3741	219	COMBINATION KEY	1	1		
12	keyboard	LALT + TAB	0	1	1	4119	00:00:04	4323	00:00:04	204	0	COMBINATION KEY	1	1		
13	focus	new 1 - Notepad++			1	7772	00:00:07	7772	00:00:07	0	0	CHANGE	1	1		
14	keyboard	LALT			1	8120	00:00:08	8326	00:00:08	206	348	COMBINATION KEY	1	1		
16	keyboard	LALT + TAB			1	8191	00:00:08	8316	00:00:08	125	0	COMBINATION KEY	1	1		
17	focus	WordLog MainDoc			1	8326	00:00:08	8326	00:00:08	0	0	CHANGE	1	1		
18	mouse	Movement			1	8625	00:00:08	8844	00:00:08	219	299	MOUSE	1	1	2644	498
19	mouse	Movement			1	12828	00:00:12	13078	00:00:13	250	3984	MOUSE	1	1	2759	459

Figure 2: Example of Cross Origin Validation workaround

4. Solution Options

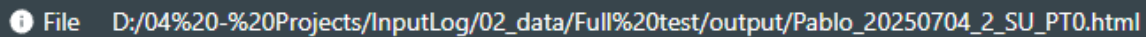
4.1 Embedding the XSL Directly into the XML

Embedding an `<xsl:stylesheet>` block within the XML and referencing it via `href="#stylesheet"` creates a single, self-contained file that users can open by double-clicking, with no external dependencies or internet required.

While this approach ensures portability and eliminates cross-origin restrictions, it increases file size, duplicates the stylesheet across multiple documents, and complicates maintenance since any style changes must be propagated into each XML file. This injection can be automated in the generation script.

4.2 Generating a Self-Contained HTML Report

An HTML report can be created by transforming the XML file (via XSLT), delivering a browser-friendly document that supports rich formatting, images, and interactive elements.

A screenshot of a file explorer window. The address bar shows the file path: "D:/04%20-%20Projects/InputLog/02_data/Full%20test/output/Pablo_20250704_2_SU_PT0.html". The file name is "Pablo_20250704_2_SU_PT0.html".

File D:/04%20-%20Projects/InputLog/02_data/Full%20test/output/Pablo_20250704_2_SU_PT0.html

Figure 3: Example of self-contained HTML report

4.3 Solution Approach

In this release, we adopted the second strategy—implementing a dedicated XML-to-HTML translation stage—detailed in section 4.2. The core change is the introduction of the `XmlToHtmlTranslator` class under the `Core.IO.Html` namespace. This component ingests the XML outputs previously written to `output/`, wraps their contents in a standardized HTML template, and emits the result as `.html` files in the same folder. After translation, the original XML files are relocated into a new `Xml/` directory to preserve them for auditing or alternative workflows.

Directly targeting the `Xml/` folder during XML generation was considered but deemed too risky: existing writer components (e.g., `GeneralAnalysisXmlWriter`) share a unified `DestinationPath` provided by `AnalyzeMode.cs`, which lacks a dedicated XML subpath. Refactoring all writers to support separate paths would require extensive changes across the codebase and introduce potential regression points. Moving files post-generation isolates risk and preserves the current architecture.

Each translation run locates the appropriate XML file by matching its base name against a configurable mapping. The translator uses a regular expression to identify the file key within the cleaned filename:

```
return keys.First(k =>
    Regex.IsMatch(cleanName, @$@"_(?i:{Regex.Escape(k)})(?=(_|\\s|\\(|\\.|$))")
);
```

This mechanism supports successive output filenames (e.g., `General_Analysis(1).xml`) without requiring manual updates. However, if analysis types or file naming conventions change, the mapping dictionary below must be updated accordingly:

```
private static Dictionary<string,string> CreateAnalysisMap() => new
Dictionary<string,string>(StringComparer.OrdinalIgnoreCase)
{
    ["BA"] = "bigram_analysis.xml",
    ["GA"] = "general_analysis.xml",
    ["GEA"] = "general_eyetrack_analysis.xml",
    ["LA"] = "linear_analysis.xml",
    ["PA"] = "pause_analysis.xml",
    ["RM"] = "revisionmatrix_analysis.xml",
    ["SN"] = "snotation_analysis.xml",
    ["SO"] = "source_analysis.xml",
    ["SU"] = "summary_analysis.xml"
};
```

All methods and logic within `XmlToHtmlTranslator` are comprehensively documented in-line, enabling developers to understand and extend the functionality without referring to external documentation.

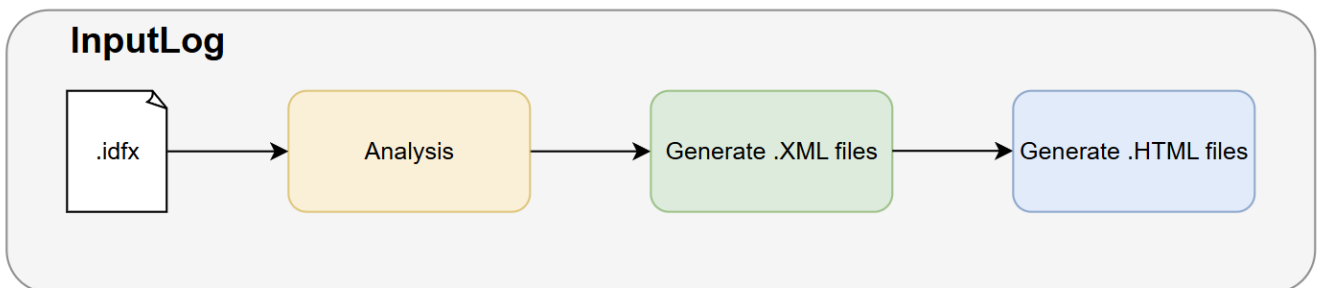


Figure 5: Process Overview

We appended the translation step at the end of the existing pipeline, ensuring no modifications to upstream analysis or XML-writing components. This non-intrusive integration preserves stability while delivering HTML outputs alongside legacy XML.

5. References

- [1] Mozilla Developer Network. [CORS Overview](#)
- [2] JavaScript.info. [Fetch and Cross-Origin Requests](#)
- [3] Mozilla Developer Network. [XSLT: Extensible Stylesheet Language Transformations](#)